

DDA5001 Machine Learning

Convex Optimization & Gradient-based Optimization Algorithm

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

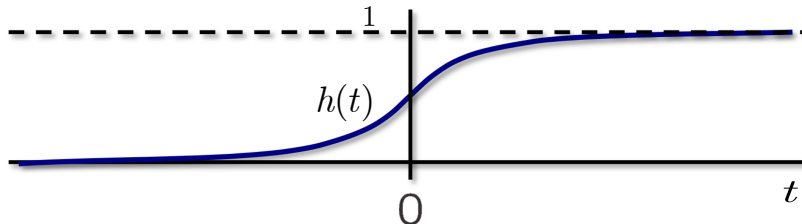


Recap: Logistic Function

The function

$$h(t) = \frac{e^t}{e^t + 1} = \frac{1}{1 + e^{-t}}$$

is called the **logistic function** or **sigmoid**.



Sigmoid: 'S'-like function.

Some other 'S'-like function: **Hyperbolic tangent**: $\tanh(t) = \frac{e^t - e^{-t}}{e^t + e^{-t}}$.

Recap: Logistic Regression for Binary Classification

Logistic Regression (LR) Model:

$$\Pr_{\boldsymbol{\theta}}[y|\mathbf{x}] = \frac{1}{1 + \exp\left(-y \cdot \boldsymbol{\theta}^{\top} \mathbf{x}\right)}$$

Through MLE principle, the learning problem of LR is given by

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^{\top} \mathbf{x}_i \right) \right)$$

- ▶ LR is for **classification**.
- ▶ LR is a **linear** classifier.

Recap: Softmax and Multi-class Logistic Regression

- ▶ Consider K classes. Assign each class $k = 1, \dots, K$ a parameter / weight vector θ_k .
- ▶ Let $\Theta = [\theta_1, \dots, \theta_K] \in \mathbb{R}^{(d+1) \times K}$ and $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be the training data.
- ▶ **Softmax:**

$$\Pr[y_i = k | \Theta, \mathbf{x}_i] = \frac{\exp(\theta_k^\top \mathbf{x}_i)}{\sum_{j=1}^K \exp(\theta_j^\top \mathbf{x}_i)}$$

- ▶ Multi-class logistic regression learning problem:

$$\hat{\Theta} = \underset{\Theta \in \mathbb{R}^{d \times K}}{\operatorname{argmin}} -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K 1_{\{y_i=k\}} \log \left(\frac{\exp(\theta_k^\top \mathbf{x}_i)}{\sum_{j=1}^K \exp(\theta_j^\top \mathbf{x}_i)} \right),$$

How to Learn $\hat{\theta}$?

The objective function (using binary logistic regression as an example)

$$\mathcal{L}(\theta) := \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \theta^\top x_i \right) \right)$$

The learning problem (from MLE principle and how to make Er_{out} small)

$$\hat{\theta} = \underset{\theta \in \mathbb{R}^d}{\operatorname{argmin}} \mathcal{L}(\theta)$$

- ▶ Bad news $\times$: No closed-form solution.
- ▶ Good news $\checkmark$: The objective function $\mathcal{L}(\theta)$ is **convex** in θ .
 - $\leadsto$ Convex optimization and gradient-based learning algorithm.

Agenda

Convex Optimization

Gradient-based Optimization Algorithms

What is Convex Optimization?

Consider the optimization problem:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta})$$

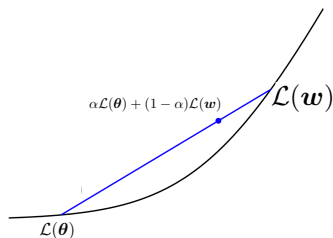
- ▶ The optimization problem is said to be **convex optimization** if $\mathcal{L}(\boldsymbol{\theta})$ is a **convex function**.
- ▶ Otherwise, it is called **nonconvex optimization**.

Definition of Convex Function

Definition: Convex function

A function $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ is **convex** if for all $\boldsymbol{\theta}, \boldsymbol{w} \in \mathbb{R}^d$ and any $\alpha \in [0, 1]$,

$$\mathcal{L}(\alpha\boldsymbol{\theta} + (1 - \alpha)\boldsymbol{w}) \leq \alpha\mathcal{L}(\boldsymbol{\theta}) + (1 - \alpha)\mathcal{L}(\boldsymbol{w})$$



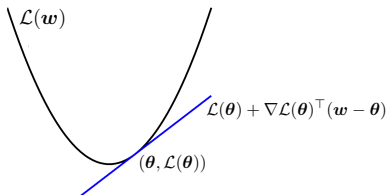
- ▶ Geometric intuition: Uniform upward curvature.
- ▶ Simple examples: $\mathcal{L}(\theta) = \theta$, $\mathcal{L}(\theta) = \theta^2$, $\mathcal{L}(\theta) = |\theta|$, $\mathcal{L}(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|^2$.

First-order Characterization of Convexity

Theorem: First order convexity characterization

Suppose $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ is **differentiable**. $\mathcal{L}$ is convex **if and only if** for all $\theta, w \in \mathbb{R}^d$

$$\mathcal{L}(w) \geq \mathcal{L}(\theta) + \nabla \mathcal{L}(\theta)^\top (w - \theta).$$

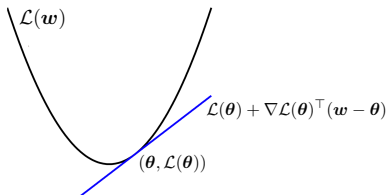


First-order Characterization of Convexity

Theorem: First order convexity characterization

Suppose $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ is **differentiable**. $\mathcal{L}$ is convex **if and only if** for all $\theta, w \in \mathbb{R}^d$

$$\mathcal{L}(w) \geq \mathcal{L}(\theta) + \nabla \mathcal{L}(\theta)^\top (w - \theta).$$



- ▶ This theorem is often used for analysis.
- ▶ Implication:

$\nabla \mathcal{L}(\theta^*) = \mathbf{0}$ if and only if θ^* is global minima.

- ▶ This is how we find the optimal parameters for Least squares (linear regression).

Second-order Characterization of Convexity

Theorem: Convexity via Hessian

Let $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ be **twice continuously differentiable**. Then $\mathcal{L}$ is convex **if and only if** its Hessian matrix is **positive semidefinite (PSD)**, i.e.,

$$\mathbf{d}^\top \nabla^2 \mathcal{L}(\boldsymbol{\theta}) \mathbf{d} \geq 0 \quad \forall \mathbf{d} \in \mathbb{R}^d, \quad \forall \boldsymbol{\theta} \in \mathbb{R}^d.$$

- A way to test convexity if the objective function is twice cont. differentiable.

Examples: Convex Instances in Machine Learning

We have the following functions are convex:

- **Least squares:**

$$\mathcal{L}(\boldsymbol{\theta}) = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2.$$

Convex because the Hessian $\nabla^2 \mathcal{L}(\boldsymbol{\theta}) = 2\mathbf{X}^\top \mathbf{X}$ is PSD.

- **Logistic regression:**

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^\top \mathbf{x}_i \right) \right).$$

- **Multi-class logistic regression:**

$$\mathcal{L}(\boldsymbol{\Theta}) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K 1_{\{y_i=k\}} \log \left(\frac{\exp(\boldsymbol{\theta}_k^\top \mathbf{x}_i)}{\sum_{j=1}^K \exp(\boldsymbol{\theta}_j^\top \mathbf{x}_i)} \right).$$

- SVM learning problem (later).

The Advantage of Convex Optimization

- ▶ No local minimum. **Zero gradient** means global optimal solution, corresponding to $\hat{\theta}$.
- ▶ Though we usually do not have closed-form solution, but we have reliable and efficient algorithms to find the global minimum, i.e., points provide zero gradient.
- ▶ There are a set of fully developed algorithmic tools for convex optimization.

Algorithms:

- ▶ Gradient-based method.
- ▶ Subgradient method.
- ▶ ...

The 'Easy' and 'Difficult' Optimization Problems

- ▶ Linear v.s. nonlinear?
- ▶ Differentiable v.s. nondifferentiable?

The 'Easy' and 'Difficult' Optimization Problems

- ▶ Linear v.s. nonlinear?
- ▶ Differentiable v.s. nondifferentiable?

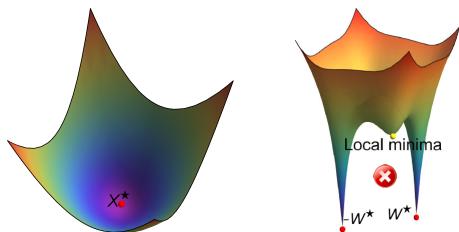


Figure: Convex geometry and nonconvex geometry.

Classify whether a problem is hard or easy: Convex (easy) v.s. nonconvex (hard).

- ▶ **Convex optimization**: Reasonable algorithms can almost always find the global minimizer, i.e., $\hat{\theta}$.
- ▶ **Nonconvex optimization**: It is very hard to find a global minimizer.

Algorithms for Learning $\hat{\theta}$

What we have so far?

- ▶ Logistic regression does not have a closed-form solution.
- ▶ Logistic regression is a convex optimization problem.
- ▶ Convex optimization problems are easy to solve.

⇒ Algorithmic tool:

Gradient-based optimization algorithms.

Agenda

Convex Optimization

Gradient-based Optimization Algorithms

Iterative Algorithm

Iterative algorithm

Start with an **initial point** θ_0 , an **iterative** algorithm $\mathcal{A}$ will generate a sequence of **iterates**

$$\theta_{k+1} = \mathcal{A}(\theta_k)$$

for $k = 0, 1, 2, \dots$

- ▶ k represents **iteration**, an indexing number.
- ▶ θ_k represents **iterate** at k -th iteration.

Iterative Algorithm

Iterative algorithm

Start with an **initial point** θ_0 , an **iterative** algorithm $\mathcal{A}$ will generate a sequence of **iterates**

$$\theta_{k+1} = \mathcal{A}(\theta_k)$$

for $k = 0, 1, 2, \dots$

- ▶ k represents **iteration**, an indexing number.
- ▶ θ_k represents **iterate** at k -th iteration.

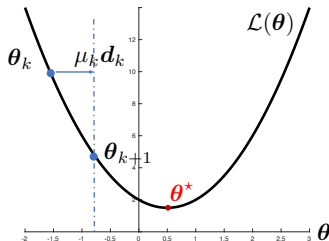
What form $\mathcal{A}$ usually has in practice?

$$\theta_{k+1} = \theta_k + \mu_k \mathbf{d}_k$$

- ▶ $\mathbb{R} \ni \mu_k > 0$ is **stepsize / learning rate**.
- ▶ $\mathbf{d}_k \in \mathbb{R}^d$ is the **search direction**, typically depends on θ_k .
- ▶ The key is to choose a proper direction $\mathbf{d}_k$ at each iteration.

Illustration and Important Elements

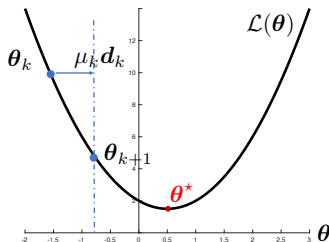
Iterative algorithm: $\theta_{k+1} = \theta_k + \mu_k d_k$.



- The new iterate θ_{k+1} is expected to be closer to θ^* than θ_k

Illustration and Important Elements

Iterative algorithm: $\theta_{k+1} = \theta_k + \mu_k d_k$.



- The new iterate θ_{k+1} is expected to be closer to θ^* than θ_k

Things to determine:

- Initial point θ_0 (fine for convex optimization).
- Search direction d_k .
- Learning rate μ_k .
- Stopping criterion.

Search Direction $\mathbf{d}_k$

Goal:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}).$$

The least:

$\mathbf{d}_k$ should point to a direction that **decreases the function value**.

Proposition: Descent direction

Suppose $\mathcal{L}$ is **continuously differentiable**, if there exists a $\mathbf{d}$ such that

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top \mathbf{d} < 0$$

then, there exists a $\tilde{\mu} > 0$ such that

$$\mathcal{L}(\boldsymbol{\theta} + \mu \mathbf{d}) < \mathcal{L}(\boldsymbol{\theta})$$

for all $\mu \in (0, \tilde{\mu})$. Thus, **$\mathbf{d}$ is a descent direction at $\boldsymbol{\theta}$** .

► The proposition can be proved by Taylor Theorem.

Gradient Descent

- This proposition tells us: At k -th iteration, find a $\mathbf{d}_k$ satisfying

$$\nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top \mathbf{d}_k < 0.$$

Then, $\mathbf{d}_k$ must be a descent direction at the current iterate $\boldsymbol{\theta}_k$.

Gradient Descent

- This proposition tells us: At k -th iteration, find a $\mathbf{d}_k$ satisfying

$$\nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top \mathbf{d}_k < 0.$$

Then, $\mathbf{d}_k$ must be a descent direction at the current iterate $\boldsymbol{\theta}_k$.

Thus, one possible choice is

$$\mathbf{d}_k = -\nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

The resultant algorithm

Gradient descent (GD)

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

Gradient Descent: Interpretation

The gradient descent

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

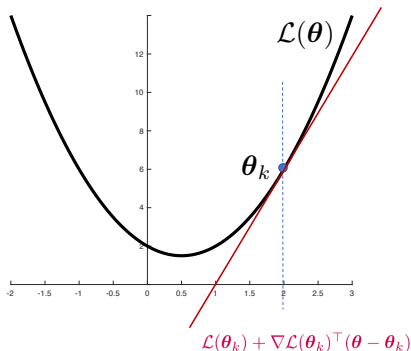
can be equivalently written as

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2$$

- ▶ $\mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k)$ is linear approximation of $\mathcal{L}$ at $\boldsymbol{\theta}_k$.
- ▶ $\frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2$ is proximal term related to learning rate μ_k .

Gradient Descent: Interpretation

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2$$



- ▶ Cannot directly minimize the linear approximation.
- ▶ The linear approximation is accurate only around θ_k .
- ▶ Thus, we need the **proximal term**.

- ▶ The **proximal term** is used to control how far the algorithm goes.

Gradient Descent: Interpretation

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ l_k(\boldsymbol{\theta}) := \mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

- ▶ $l_k(\boldsymbol{\theta})$ is a quadratic function in $\boldsymbol{\theta}$, i.e., a quadratic approximation
- ▶ At each iteration, we have closed-form update, i.e., the gradient descent algorithm

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

Gradient Descent: Interpretation

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ l_k(\boldsymbol{\theta}) := \mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

- ▶ $l_k(\boldsymbol{\theta})$ is a quadratic function in $\boldsymbol{\theta}$, i.e., a quadratic approximation
- ▶ At each iteration, we have closed-form update, i.e., the gradient descent algorithm

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

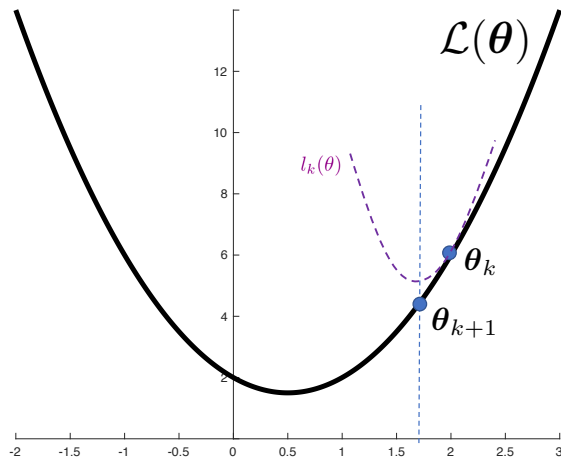
Almost universal algorithmic design strategy

Solving the original problem by solving a sequence of simpler subproblems.

The simpler subproblem can be interpreted in two ways:

1. A **linear approximation** of $\mathcal{L}(\boldsymbol{\theta})$ + a **proximal term**
2. A quadratic approximation $l_k(\boldsymbol{\theta})$ of $\mathcal{L}(\boldsymbol{\theta})$

Gradient Descent: Interpretation



- Iteratively construct $l_k(\theta)$ to get the next θ_{k+1}

A Useful Algorithm Design Framework

Suppose the task is $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta})$, we can design an algorithm as

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ q_k(\boldsymbol{\theta}) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

μ_k is learning rate-like quantity.

A Useful Algorithm Design Framework

Suppose the task is $\min_{\theta \in \mathbb{R}^d} \mathcal{L}(\theta)$, we can design an algorithm as

$$\theta_{k+1} = \operatorname{argmin}_{\theta \in \mathbb{R}^d} \left\{ q_k(\theta) + \frac{1}{2\mu_k} \|\theta - \theta_k\|_2^2 \right\}$$

μ_k is learning rate-like quantity.

- ▶ When $q_k(\theta)$ is **linear approximation** of $\mathcal{L} \implies$ **gradient descent**
- ▶ When $q_k(\theta)$ is **second-order approximation** of $\mathcal{L} \implies$ **Newton's method**
- ▶ When $q_k(\theta)$ is $\mathcal{L}$ itself $\implies$ **proximal point method**
- ▶ When $q_k(\theta)$ is **single component linear approximation** of $\mathcal{L} \implies$ **stochastic gradient descent (SGD)**
- ▶ ...

Many optimization algorithms follow this designing framework.

▪

Convergence Issue

Convergence of Iterative Algorithm

- ▶ To solve $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta})$, we cannot obtain the solution $\hat{\boldsymbol{\theta}}$ analytically.
- ▶ Design an iterative algorithm, start with $\boldsymbol{\theta}_0$, it will generate

$$\{\boldsymbol{\theta}_0, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \dots, \boldsymbol{\theta}_k, \dots\}.$$

Convergence of Iterative Algorithm

- ▶ To solve $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta})$, we cannot obtain the solution $\hat{\boldsymbol{\theta}}$ analytically.
- ▶ Design an iterative algorithm, start with $\boldsymbol{\theta}_0$, it will generate

$$\{\boldsymbol{\theta}_0, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \dots, \boldsymbol{\theta}_k, \dots\}.$$

Convergence analysis of an algorithm concerns:

- ▶ Will $\boldsymbol{\theta}_k$ converge to the solution $\hat{\boldsymbol{\theta}}$? That is

$$\lim_{k \rightarrow \infty} \boldsymbol{\theta}_k \stackrel{?}{=} \hat{\boldsymbol{\theta}}.$$

- ▶ If yes, what is the speed of this convergence?

Convergence of GD

- Suppose that $\mathcal{L}$ is **convex** and **differentiable** and has **Lipschitz continuous gradient** with parameter L ,

$$\|\nabla\mathcal{L}(\mathbf{w}) - \nabla\mathcal{L}(\mathbf{u})\|_2 \leq L\|\mathbf{w} - \mathbf{u}\|_2, \quad \forall \mathbf{w}, \mathbf{u}$$

↪ Both convexity and Lipschitz gradient are satisfied in LR.

Theorem: Convergence and Convergence rate of GD

Gradient descent with constant learning rate $\mu_k = \mu = 1/L$ satisfies

$$\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}(\hat{\boldsymbol{\theta}}) \leq \frac{L\|\boldsymbol{\theta}_0 - \hat{\boldsymbol{\theta}}\|_2^2}{2k}$$

- $\mathcal{L}(\boldsymbol{\theta}_k)$ converges to $\mathcal{L}(\hat{\boldsymbol{\theta}})$ at the rate of $\mathcal{O}(1/k)$.
- It does not mean $\{\boldsymbol{\theta}_k\}$ converges to $\hat{\boldsymbol{\theta}}$ at a certain rate.
- $\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}(\hat{\boldsymbol{\theta}})$ is called **sub-optimality gap**.

↪ Next lecture: More on GD